



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського» Факультет інформатики та обчислювальної
техніки
Кафедра інформатики і програмної інженерії

Звіт
з дисципліни «Компоненти програмної інженерії»
Лабораторна робота №5
"Масштабування системи"

Виконав:

Студент II курсу
гр. ІП-33
Соколов О. В.

Перевірів:

Зарічковий О. А.

Лабораторні робота №5.

Масштабування системи

Тема: Масштабування системи

Мета: Масштабувати кожен елемент системи.

Постановка задачі:

1. Зробити високорівневий capacity planning для кожного елемента системи (мікросервісу, бази даних, системи обміну повідомленнями тощо):

а. вказати навантаження, наприклад: скільки запитів може обслуговувати один інстанс мікросервісу, яке навантаження потягне база даних тощо

б. вказати відповідну конфігурацію сервера під навантаження

с. вказати вартість

2. Вкажіть для кожного елемента системи, який тип масштабування буде використовуватись в кожному випадку:

а. мікросервіси

б. система обміну повідомленнями

с. кеш (якщо є)

d. база даних

е. система потокової обробки даних (якщо є).

3. Вкажіть причини, з яких ви застосовуєте масштабування взагалі і кожний тип масштабування зокрема. Будьте дуже точними та базуйтеся на прикладі своєї системи та ваших кейсів.

4. Вказати для кожного елемента системи, як відбувається балансування навантаження у випадку горизонтального масштабування (як відбувається диспетчеризація)

Виконання

1. Високорівневе планування потужностей (capacity planning)

Система має забезпечити 100 000 одночасно активних користувачів, які слухають музику або подкасти у режимі потокового передавання.

Таблиця 1 – Capacity planning

Компонент (сервіс)	Навантаження (на 1 інстанс)	Рекомендована конфігурація сервера	Орієнтовна вартість (на місяць)
Streaming Service	до 1000 одночасних потоків	2 CPU, 4 GB RAM	t3.medium ≈ \$33
User Service	до 500 запитів/с	2 CPU, 2 GB RAM	t3.small ≈ \$17
Search Service	до 200 запитів/с	2 CPU, 4 GB RAM	t3.medium ≈ \$33
Recommendation Service	до 100 запитів/с	2 CPU, 4 GB RAM	t3.medium ≈ \$33
Ads Service	до 150 запитів/с	1 CPU, 2 GB RAM	t3.small ≈ \$17
Email Service	до 10-20 запитів/с	1 CPU, 1 GB RAM	t3.micro ≈ \$8
Playlists Service	до 300 запитів/с	2 CPU, 2 GB RAM	t3.small ≈ \$17
Analytics Service	до 500 подій/с	2 CPU, 4 GB RAM	t3.medium ≈ \$33
Payments Service	до 100 транзакцій/с	2 CPU, 4 GB RAM	t3.medium ≈ \$33
PostgreSQL	до 5000 TPS	2 vCPU, 8 GB RAM	db.m5.large ≈ \$100
Redis	до 10 000 запитів/с	cache.t3.medium (2GB RAM)	≈ \$40
Kafka	до 50 000 повідомлень/с	3 брокери, 2 vCPU, 8 GB RAM	≈ \$300
Apache Flink	до 10 000 подій/с	2 task manager, 4 vCPU, 8 GB RAM	≈ \$150
AWS S3 + CloudFront	до 100 000 потоків	S3 + CDN	≈ \$300

2. Типи масштабування системи

У рамках архітектури стрімінгового сервісу, орієнтованого на обслуговування 100 000 активних користувачів, для кожного компоненту системи визначено оптимальний тип масштабування, виходячи з характеру навантаження, вимог до продуктивності та доступності.

2.1 Мікросервіси

Тип масштабування: горизонтальне

Пояснення: Мікросервіси, зокрема Streaming Service, User Service, Search Service, Recommendation Service, Playlists Service, Payments Service та інші, є незалежними компонентами, що обробляють HTTP-запити або події. Їхня логіка дозволяє масштабувати їх горизонтально — додаванням нових інстансів під балансувальник навантаження, що забезпечує високий рівень відмовостійкості та масштабованості.

2.2 Обмін повідомленнями (kafka)

Тип масштабування: горизонтальне (кластерне масштабування брокерів)

Пояснення: Kafka дозволяє масштабуватися за рахунок додавання нових брокерів до кластера. Це забезпечує підвищення пропускної здатності, а також розподілення партицій топіків між брокерами.

2.3 Кеш (Redis)

Тип масштабування: вертикальне (основний), з можливістю шардінгу для горизонтального

Пояснення: Redis як in-memory сховище найчастіше масштабують вертикально (збільшення пам'яті або CPU). Для великих навантажень

можливе горизонтальне масштабування через Redis Cluster (шардінг по ключах), але це потребує додаткового конфігурування.

2.4 База даних (PostgreSQL)

Тип масштабування: вертикальне (для основного інстансу) + реплікація (горизонтально для читання)

Пояснення: Основний інстанс PostgreSQL масштабують вертикально для забезпечення стабільної роботи записів. Для зменшення навантаження на основну БД реалізується горизонтальне масштабування на рівні читання — через реплікацію та read-replicas (наприклад, Amazon Aurora PostgreSQL).

2.5 Поточкова обробка даних (Apache Flink)

Тип масштабування: горизонтальне (кластерне)

Пояснення: Flink дозволяє масштабування за рахунок збільшення кількості task manager'ів у кластері. Завдяки цьому можлива паралельна обробка потоків подій, що є критично важливим при високому навантаженні (наприклад, реальний час аналізу track_played).

3. Причини застосування масштабування

Масштабування є ключовим принципом побудови надійної та ефективної системи, яка повинна обслуговувати **щонайменше 100 000 одночасно активних користувачів**. У межах архітектури музичного стрімінгового сервісу масштабування дозволяє забезпечити:

- високу доступність сервісів;
- стабільну роботу при піковому навантаженні;
- зменшення затримок у відтворенні контенту;
- обробку великого обсягу подій та аналітики у реальному часі;
- уникнення простою при збої окремих компонентів.

3.1. Мікросервіси — горизонтальне масштабування

Чому: Мікросервіси обробляють запити користувачів, тому при зростанні трафіку (наприклад, у вечірній час) кількість запитів може збільшуватись у кілька разів. Зокрема, Streaming Service повинен підтримувати тисячі одночасних потоків. Горизонтальне масштабування дозволяє запуснути додаткові інстанси без модифікації коду.

Приклад: Під час релізу нового альбому популярного артиста система може прийняти одночасно 10 000 запитів до одного треку — балансувальник автоматично розподіляє навантаження між кількома інстансами.

3.2. Kafka (система повідомлень) — горизонтальне масштабування

Чому: У системі генерується велика кількість подій: track_played, track_liked, payment_successful. Kafka дозволяє розподіляти ці події між кількома брокерами, підвищуючи пропускну здатність і запобігаючи затримкам.

Приклад: Кожне відтворення треку створює подію. Якщо 100 000 користувачів слухають музику одночасно, система повинна обробити десятки тисяч подій щосекунди.

3.3. Redis (кеш) — вертикальне масштабування (з можливістю шардінгу)

Чому: Кеш використовується для зберігання часто доступних даних — наприклад, списків рекомендацій або профілів користувачів. Redis працює в оперативній пам'яті, тому критично важливо мати достатній об'єм RAM. Масштабування Redis відбувається переважно вертикально — із можливістю горизонтального розширення через Redis Cluster.

Приклад: Коли користувач відкриває застосунок, список рекомендованих треків швидко витягується з Redis — зменшуючи навантаження на основну базу.

3.4. PostgreSQL — вертикальне (для запису) та горизонтальне (для читання)

Чому: Основні транзакції — створення плейлистів, запис історії відтворень, оновлення профілю — потребують стабільної роботи основної БД. Запити на читання (перегляд списку треків, даних користувача) масштабуються за допомогою read-replicas.

Приклад: Коли 50 000 користувачів одночасно переглядають свій профіль або плейлисти, ці запити розподіляються на репліки, знижуючи навантаження на головну БД.

3.5. Apache Flink — горизонтальне масштабування

Чому: Система потокової обробки використовується для обчислення реального рейтингу треків, моніторингу прослуховувань, обробки поведінки користувачів. При зростанні кількості подій потрібне додавання нових обробників подій (task manager'ів).

Приклад: При масовому прослуховуванні нової пісні Flink оперативно підраховує кількість відтворень кожні 5 секунд і оновлює live-рейтинг треків.

4. Механізми балансування навантаження при горизонтальному масштабуванні

У межах реалізації горизонтального масштабування кожного з компонентів системи застосовуються різні підходи до диспетчеризації запитів і балансування навантаження. Нижче наведено способи реалізації балансування для кожного елементу архітектури.

4.1. Мікросервіси

Механізм балансування: HTTP-запити маршрутизуються через **Ingress Controller** у Kubernetes (наприклад, NGINX або Traefik), який виступає в ролі балансувальника навантаження. Kubernetes Service використовує **round-robin** або **least connection** для розподілу запитів між подами одного сервісу.

Приклад: Коли 10 інстансів Streaming Service одночасно обслуговують трафік, Ingress Controller рівномірно розподіляє запити між ними.

4.2. Kafka (система обміну повідомленнями)

Механізм балансування: Kafka використовує **партиції** топіків, які розподіляються між **брокерами**. Кожен consumer підписується на певну партицію, і Kafka автоматично балансує навантаження по consumer-групі.

Приклад: Якщо 3 брокери Kafka обробляють 6 партицій топіка track_played, кожен з них отримує по 2 партиції, а споживачі з consumer group обробляють повідомлення паралельно.

4.3. Redis (кеш)

Механізм балансування: У Redis Cluster застосовується **шардінг ключів**: ключі хешуються та розподіляються по кількох нодах. Балансування відбувається автоматично на рівні клієнтської бібліотеки, яка знає, в яку ноду звертатися.

Приклад: Кешовані плейлисти користувачів розподіляються за `user_id` — кожен вузол Redis обслуговує частину даних, що зменшує затримки при читанні.

4.4. PostgreSQL (БД)

Механізм балансування: Для реплік (read replicas) балансування здійснюється через **middleware** (наприклад, PgPool-II або ProxySQL) або на рівні програми — різні запити читають з різних реплік.

Приклад: Запити на перегляд профілю користувача надсилаються на одну з трьох read-replicas, тоді як запити на оновлення даних — на головний екземпляр.

4.5. Apache Flink (stream processing)

Механізм балансування: Flink автоматично розподіляє навантаження за допомогою **task slots** у task manager'ах. Дані партиціонуються (keyBy) і обробляються паралельно відповідно до ключів.

Приклад: Обробка подій `track_played` виконується паралельно — кожен task slot відповідає за окрему групу треків (за `track_id`).

Висновок:

У ході виконання лабораторної роботи було проведено високорівневе планування потужностей для всіх компонентів стрімінгового сервісу, призначеного для обслуговування до 100 000 одночасно активних користувачів. Було визначено навантаження на кожен мікросервіс, базу даних, систему кешування, брокер повідомлень та платформу потокової обробки, а також підібрано оптимальні конфігурації серверів і орієнтовну вартість їх використання у хмарному середовищі.

Для кожного компоненту системи було обґрунтовано відповідний тип масштабування. Основні мікросервіси масштабуються горизонтально з використанням Kubernetes та Ingress Controller, що забезпечує гнучке та надійне балансування навантаження. Kafka та Flink масштабуються кластерно — за рахунок розподілу партицій і задач між брокерами та виконавцями. Redis передбачає переважно вертикальне масштабування із можливістю використання шардінгу. PostgreSQL масштабовано вертикально для обробки транзакцій та горизонтально за рахунок використання read-replicas для читання.

Було чітко сформульовано причини застосування кожного типу масштабування, виходячи з практичних сценаріїв роботи системи — таких як велика кількість одночасних потоків, висока частота запитів до кешу, потреба в обробці подій у реальному часі, або навантаження на базу даних. Для кожного випадку було описано механізм балансування — від HTTP-роутингів до розподілу Kafka-партицій і Flink task slot'ів.

Загалом робота підтвердила, що правильно обране масштабування є критично важливим для побудови стабільної, високопродуктивної, відмовостійкої та масштабованої архітектури сучасного стрімінгового сервісу.